COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/28/2026	**DATE SUBMITTED:**	
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

Code

main.cpp

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "searchingAlgo.h"

const int max_size = 50;

int main() {


// Linear Search Array

    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        dataset[i]=rand();
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;
```

Screenshot

```
bugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-zctg5dj3.mjk' '--stdout=Microsoft-MIEngine-Out-khlp0fhw.k00' '--stder
r=Microsoft-MIEngine-Error-425qcvh5.zzp' '--pid=Microsoft-MIEngine-Pid-iiai2dc4.2ae' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi
'
8513 25960 5755 10264 27759 15571 6302 28354 32380 27857 3448 23553 17564 7446 28664 26838 22113 16189 607 18766 26416 2665 27473 25933 27634
23085 10540 20106 18699 29865 20378 22491 7312 31734 17250 12723 30900 3781 26756 12264 8836 23618 30340 7354 10453 12095 26374 15634 26490 13
454
```

Observation

The program successfully produced and displayed 50 random integer numbers using the rand() function. The numbers produced vary each time the program is run due to the srand(time(0)) function.

Table 6-1. Data Generated and Observations.

Code

main.cpp

```
20
21      int choice;
22      std::cout << "Indicate the number to search: ";
23      std::cin >> choice;
24
25      linearSearchArr(max_size, choice, dataset);
```

searchingAlgo.h

```
// Linear Search Array
template <typename T>
void linearSearchArr(int maxSize, T dataFind, T data[]){
    int i = 0;
    while(i <= maxSize){
        if(dataFind == data[i]){
            std::cout << "Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout << "the search is unsuccessful\n";

}
```

Screenshot

```
Indicate the number to search: 1
the search is unsuccessful
```

Observation

Linear search compares each element of the array until the search item is found. If the item is found, it prints “Searching is successful.” Else, it prints “Searching is unsuccessful.”

Table 6-2a. Linear Search for Arrays

Code

main.cpp

```
28
29  // Linear Search Linked List
30
31      Node<char>* name1 = new_node('R');
32      Node<char>* name2 = new_node('A');
33      Node<char>* name3 = new_node('L');
34      Node<char>* name4 = new_node('P');
35      Node<char>* name5 = new_node('H');
36
37      name1->next = name2;
38      name2->next = name3;
39      name3->next = name4;
40      name4->next = name5;
41
42      linearSearchLL(name1, 'R');
43      linearSearchLL(name2, 'D');
44
```

searchingAlgo.h

```
50  template <typename T>
51  void linearSearchLL(Node<T> *node, T dataFind) {
52      while(node != nullptr){
53
54          if(node->data == dataFind){
55              std::cout << "the search was successful.\n";
56              return;
57          }
58          node = node->next;
59      }
60      std::cout << "the search was unsuccessful\n";
61  }
62
```

Screenshot

```
the search was successful.
the search was unsuccessful
```

Observation

Nodes are inspected one by one starting from the start node till the required value is found or till the last node of the linked list is reached in the process of linear search algorithm. The search is a success for 'R', however, 'D' cannot be found since it is not present in the search list.

Table 6-2b. Linear Search for Linked List

Code

main.cpp

```
49      int dataset2[max_size];
50      for(int i = 0; i < max_size; i++){
51          dataset2[i] = i;
52          std::cout << dataset2[i] << " ";
53      }
54
55      binarySearchArr(11, dataset2, max_size);
56      binarySearchArr(11111, dataset2, max_size);
```

searchingAlgo.h

```
66  template <typename T>
67  void binarySearchArr(T dataFind, T data[], int maxSize){
68      int low = 0, up = maxSize-1, middle;
69      while(low <= up){
70          middle = (int)((low+up)/2);
71          if(dataFind == data[middle]){
72              std::cout << "\nSearching is successful\n";
73              return;
74          }
75          else if(dataFind > data[middle]){
76              low = middle + 1;
77          }else {
78              up = middle - 1;
79          }
80      }
81      std::cout << "Search element was not found.\n";
82  }
```

Screenshot

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
Searching is successful
Search element was not found.
```

Observation

The binary search algorithm searches for the number 11 in the given sorted list. The search algorithm cannot find the number 11111 because the number is not present in the list. The algorithm works in such a way that it divides the list into two parts.

Table 6-3a. Binary Search for Arrays

Code

main.cpp

```
Node<char>* name6 = new_node('A');
Node<char>* name7 = new_node('B');
Node<char>* name8 = new_node('C');
Node<char>* name9 = new_node('D');
Node<char>* name10 = new_node('E');

name6->next = name7;
name7->next = name8;
name8->next = name9;
name9->next = name10;

binarySearchLL(name6, 'P');
binarySearchLL(name6, 'C');
```

searchingAlgo.h

```
84  template <typename T>
85  Node<T>* getMiddle(Node<T>* start, Node<T>* last){
86
87      Node<T>* slow = start;
88      Node<T>* fast = start;
89
90      while(fast != last && fast->next != last){
91          slow = slow->next;
92          fast = fast->next->next;
93      }
94      return slow;
95  }
96
97
98  // Binary Search Linked List
99
100 template <typename T>
101 void binarySearchLL(Node<T>* start, T dataFind){
102
103     Node<T>* last = nullptr;
104
105     while (start != last) {
106         Node<T>* middle = getMiddle(start, last);
107
108         if(middle->data == dataFind){
109             std::cout << "The data was found.\n";
110             return;
111         }
112         else if(middle->data > dataFind){
113             last = middle;
114         }
115         else {
116             start = middle->next;
117         }
118     }
119     std::cout << "Search element was not found.\n";
120 }
121
```

Screenshot

```
Search element was not found.
The data was found.
```

Observation

The binary search efficiently searches for 'C' within the sorted linked list. However, when it tries to search for 'P' was not successful since 'P' does not exist in the linked list. The getMiddle() function uses two pointers: a slow pointer and a fast pointer, thus helping split/divide the searching range.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the Questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem

Problem 1 CODE:

main.cpp

```
76   // ILOB PROBLEM 1
77       std::cout << "\n| Problem 1 |\n";
78
79       int data1[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
80
81       int arrayCount = linearSearchComparison(10, 18, data1);
82       std::cout << "Array comparisons for 18: " << arrayCount << std::endl;
83
84       Node<int>* node1 = new_node(15);
85       Node<int>* node2 = new_node(18);
86       Node<int>* node3 = new_node(2);
87       Node<int>* node4 = new_node(19);
88       Node<int>* node5 = new_node(18);
89       Node<int>* node6 = new_node(0);
90       Node<int>* node7 = new_node(8);
91       Node<int>* node8 = new_node(14);
92       Node<int>* node9 = new_node(19);
93       Node<int>* node10 = new_node(14);
94
95       node1->next = node2;
96       node2->next = node3;
97       node3->next = node4;
98       node4->next = node5;
99       node5->next = node6;
100      node6->next = node7;
101      node7->next = node8;
102      node8->next = node9;
103      node9->next = node10;
104
105      int listCount = linearSearchLLComparison(node1, 18);
106      std::cout << "Linked list comparisons for 18: " << listCount << std::endl;
107
```

searchingAlgo.h

```
123  // ILOB Problem 1: Count the number of comparisons
124
125  template <typename T>
126  int linearSearchComparison(int maxSize, T dataFind, T data[]){
127      int i = 0;
128      int comparisons = 0;
129      while(i < maxSize){
130          comparisons++;
131          if(dataFind == data[i]){
132              return comparisons;
133          }
134          i++;
135      }
136      return comparisons;
137  }
138
139
140  template <typename T>
141  int linearSearchLLComparison(Node<T> *node, T dataFind){
142      int comparisons = 0;
143      while(node != nullptr){
144          comparisons++;
145          if(node->data == dataFind){
146              return comparisons;
147          }
148          node = node->next;
149      }
150      return comparisons;
151  }
152
```

Problem 1 OUTPUT:

```
| Problem 1 |
Array comparisons for 18: 2
Linked list comparisons for 18: 2
```

Answer to Problem 1:

The number of comparisons in this case is 2 for both the array and the linked list. In the process of linear searching, the search starts from the first element and goes on comparing each element one after the other. As 18 is the second element, therefore, 2 comparisons will be required to find 18. I did so by implementing the linearSearchComparison() function for the array and linearSearchLLComparison() function for the linked list. Each of these functions makes one more comparison every time an element is compared.

Problem 2 CODE:

main.cpp

```
108    // ILOB PROBLEM 2
109
110        std::cout << "\n| Problem 2 |\n";
111        int total18 = linearSearchCount(10, 18, data1);
112        std::cout << "Number of 18s: "<< total18 << std::endl;
```

searchingAlgo.h

```
// ILOB Problem 2: Count repeating instances of the search key
template <typename T>
int linearSearchCount(int maxSize, T dataFind, T data[]){
    int i = 0;
    int count = 0;
    while(i < maxSize){
        if(data[i] == dataFind){
            count++;
        }
        i++;
    }
    return count;
}
```

Problem 2 OUTPUT:

```
| Problem 2 |
Number of 18s: 2
```

Answer to Problem 2:

There are 2 instances of 18 that are in the array. I implemented this by modifying the sequential search so that instead of stopping when it finds the first 18, it continues searching through the entire list. As soon as there is an occurrence of 18, the counter increases by 1, and once it reaches the end of the list, it returns the value of 2.

Problem 3 CODE:

main.cpp

```cpp
// ILOB PROBLEM 3

    std::cout << "\n| Problem 3 |\n";
    int data3[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    binarySearchSteps(data3, 0, 9, 8);
```

searchingAlgo.h

```cpp
// ILOB Problem 3: Show each step of Binary Search
template <typename T>
void binarySearchSteps(T data[], int low, int up, T dataFind){
    int middle;
    int step = 1;

    while(low <= up){
        middle = (int)((low + up) / 2);

        std::cout << "Step " << step << ": ";
        std::cout << "Low = " << low;
        std::cout << ", Middle = " << middle;
        std::cout << ", Up = " << up;
        std::cout << ", Value = " << data[middle] << std::endl;

        if(dataFind == data[middle]){
            std::cout << "Key " << dataFind << " was found at index " << middle << ".\n";
            return;
        }
        else if(dataFind > data[middle]){
            low = middle + 1;
        }
        else {
            up = middle - 1;
        }

        step++;
    }

    std::cout << "Search element was not found.\n";
}
```

Problem 3 OUTPUT:

```
| Problem 3 |
Step 1: Low = 0, Middle = 4, Up = 9, Value = 11
Step 2: Low = 0, Middle = 1, Up = 3, Value = 5
Step 3: Low = 2, Middle = 2, Up = 3, Value = 6
Step 4: Low = 3, Middle = 3, Up = 3, Value = 8
Key 8 was found at index 3.
```

Answer to Problem 3:

The number 8 occurs in index 3 in 4 stages. This was made possible through the use of the binary search method on the sorted array. The program initially searches for the middle element, which is 11. Number 8 is less than 11; hence the program moves on to the left side. Then comes the process of checking 5, 6, and eventually 8. In addition, I have provided output values for Low, Middle, Up, and Value to show the general process how it flows in the program.

DIAGRAM FOR PROBLEM 3



Problem 4 CODE:

main.cpp

```
122  // ILOB PROBLEM 4
123
124      std::cout << "\n| Problem 4 |\n";
125
126      int data4[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
127      int searchResult = biSearchRec(data4, 0, 9, 8);
128
129      if(searchResult != -1){
130          std::cout << "The data was found at index "<< searchResult << ".\n";
131      }
132      else {
133          std::cout << "Search element was not found.\n";
134      }
135
```

searchingAlgo.h

```cpp
// ILOB Problem 4: Recursive Binary Search
template <typename T>
int biSearchRec(T data[], int low, int up, T dataFind){
    if(low > up){
        return -1;
    }
    int middle = (int)((low + up) / 2);
    if(dataFind == data[middle]){
        return middle;
    }
    else if(dataFind > data[middle]){
        return biSearchRec(data, middle + 1, up, dataFind);
    }
    else {
        return biSearchRec(data,low,middle - 1,dataFind);
    }
}
```

Problem 4 OUTPUT:

```
| Problem 4 |
The data was found at index 3.
```

Answer to Problem 4:

The number 8 is located at index 3. This is achieved through modification of the binary search algorithm into a recursive function named biSearchRec() (I would name this better but it was quite a long function name). Rather than performing a loop iteration, the function invokes itself based on whether the search key is greater or smaller than the middle number. The function terminates and gives the index once the target number 8 is found. In case the search key becomes out of bounds, -1 is returned.

Conclusion:

In this hands-on activity, I performed some searching algorithms and their implementation in C++. The implementation of sequential search algorithm for an array and linked list data structure showed that the number 18 required 2 comparisons. After that, I performed the sequential algorithm in a different way and determined the frequency of the number 18, which is 2 times. During binary search, I understood how the values of low, middle, and up change and drew a picture describing how to find 8. At last, I implemented the binary search algorithm using recursion and found the value 8 with index 3.

The issue with the implementation of algorithms in C++ was that I could not comprehend the logic of binary search and recursion and implement it in C++. Therefore, I can say that I still need to learn C++ language and algorithms. I should implement more exercises, improve my skills and know not only the result but the process of its implementation. However, despite the complexity, the gradual solution of each problem helped me to

understand better searching algorithms and made me appreciate this activity more.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini, and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, formatting, summarization, code assistance, and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for concepts and learning how to implement the code, most of the code was referenced from the procedure.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.